

JAVA PROGRAMMING USING LINUX

2-MARK QUESTIONS *generated by ChatGPT*

1. Differentiate between numeric literal and non-numeric literal.

A numeric literal is a number that represents a constant value directly in code, such as 123, 45.67, or 0xAB (hexadecimal). Non-numeric literals are constants that represent non-numeric values, such as characters ('a'), strings ("hello"), and boolean values (true or false).

Example:

```
java
Copy code
int num = 100; // numeric literal
char letter = 'A'; // non-numeric literal
```

2. Differentiate between the operators = and ==.

The = operator is used for assignment, i.e., it assigns a value to a variable. The == operator is used for comparison, i.e., it checks if two values are equal.

Example:

```
java
Copy code
int a = 5; // '=' assigns value 5 to a
if (a == 5) { // '==' checks if a is equal to 5
    System.out.println("a is 5");
}
```

3. What is the use of this keyword?

The this keyword refers to the current object within a method or constructor. It is used to distinguish between instance variables and parameters with the same name or to invoke other constructors of the same class.

Example:

java

Copy code

```
class Person {
    String name;

    Person(String name) {
        this.name = name; // 'this' refers to the instance variable
    }
}
```

4. What is constructor overloading?

Constructor overloading occurs when a class has multiple constructors with different parameter lists. This allows creating objects in different ways depending on the number or type of arguments passed.

Example:

java

Copy code

```
class Person {
    String name;
    int age;

    // Constructor with one parameter
    Person(String name) {
        this.name = name;
    }

    // Constructor with two parameters
    Person(String name, int age) {
        this.name = name;
        this.age = age;
    }
}
```

5. What is a derived class?

A derived class (also known as a subclass) is a class that inherits properties and behaviors from another class, called the base class (superclass). The derived class can extend or modify the functionality of the base class.

Example:

```
java
Copy code
class Animal {
    void sound() {
        System.out.println("Animal makes a sound");
    }
}

class Dog extends Animal {
    void sound() {
        System.out.println("Dog barks");
    }
}
```

6. Define a Package.

A package in Java is a namespace that organizes classes and interfaces logically. It helps in avoiding name conflicts and can group related classes and interfaces together.

Example:

```
java
Copy code
package mypackage;

class MyClass {
    // Class code here
}
```

7. What is exception handling?

Exception handling in Java is a mechanism to handle runtime errors, allowing the program to continue its execution without crashing. It uses try, catch, throw, and finally blocks to catch and manage exceptions.

Example:

```
java
Copy code
try {
    int result = 10 / 0; // Division by zero
} catch (ArithmeticException e) {
    System.out.println("Error: Division by zero");
} finally {
    System.out.println("This block runs regardless of exception.");
}
```

8. What is Delegation event model?

The Delegation event model is used in Java's GUI programming to handle events. In this model, an event listener (delegated) is registered to listen for events like clicks or key presses, and the listener delegates the event handling to methods defined in the class.

Example:

```
java
Copy code
class ButtonClickListener implements ActionListener {
    public void actionPerformed(ActionEvent e) {
        System.out.println("Button clicked");
    }
}
```

9. Write a short note on JCheckBox.

JCheckBox is a Swing component that allows the user to choose one or more options from a set. It provides a checkbox with two states: selected (checked) and unselected (unchecked).

Example:

```
java
Copy code
JCheckBox checkBox = new JCheckBox("Accept Terms and Conditions");
checkBox.setSelected(true); // Set checkbox as checked
```

10. Define a Layout Manager. List different Layout Managers.

A Layout Manager in Java is an interface used to arrange components in a container (like a panel or window). Some of the common layout managers include FlowLayout, BorderLayout, GridLayout, BoxLayout, and CardLayout.

Example:

```
java
Copy code
setLayout(new FlowLayout()); // Setting FlowLayout for the
container
```

11. Distinguish between init() and start() methods of an applet.

In an applet, the `init()` method is called once when the applet is loaded, used for initialization, such as setting up resources. The `start()` method is called each time the applet becomes visible or starts, useful for tasks like starting animations or threads.

12. What are the JDBC API components?

The JDBC (Java Database Connectivity) API components include `DriverManager`, `Connection`, `Statement`, `ResultSet`, and `SQLException`. These components allow Java programs to connect and interact with databases.

13. Differentiate between the operators & and &&.

The & operator is a bitwise AND operator that works on individual bits, while && is a logical AND operator that evaluates expressions and returns true only if both operands are true.

Example:

```
java
Copy code
// Bitwise AND
int result1 = 5 & 3; // 0101 & 0011 = 0001

// Logical AND
boolean result2 = (5 > 3) && (8 < 10); // true
```

14. Write the syntax of switch case structure.

The switch statement is used to execute one out of many possible blocks of code based on the value of a variable.

Example:

```
java
Copy code
int day = 3;
switch(day) {
    case 1: System.out.println("Monday"); break;
    case 2: System.out.println("Tuesday"); break;
    case 3: System.out.println("Wednesday"); break;
    default: System.out.println("Invalid day");
}
```

15. Define classes and objects.

In Java, a **class** is a blueprint or template for creating objects, which defines properties (variables) and methods (functions) that the objects will have. An **object** is an instance of a class, representing a specific entity that is created from the class template.

Example:

```

java
Copy code
class Car {
    String brand;
    int year;

    void displayInfo() {
        System.out.println(brand + " " + year);
    }
}

public class Main {
    public static void main(String[] args) {
        Car car1 = new Car(); // Object created from class Car
        car1.brand = "Toyota";
        car1.year = 2020;
        car1.displayInfo();
    }
}

```

16. Why multiple inheritance is not supported in Java?

Java does not support multiple inheritance directly through classes to avoid complexity and ambiguity. If a class were to inherit from two classes that have the same method, it would cause confusion. However, Java supports multiple inheritance through interfaces, which avoids such issues.

17. What is an abstract method?

An **abstract method** is a method that is declared in an abstract class or interface but does not have a body. The actual implementation of the method is provided by the subclass.

Example:

```

java
Copy code
abstract class Animal {
    abstract void sound(); // Abstract method without

```

```

implementation
}

class Dog extends Animal {
    void sound() {
        System.out.println("Bark");
    }
}

```

18. Explain one-dimensional array with example.

A **one-dimensional array** is a data structure that stores a sequence of elements of the same type in a single line. You can access each element by its index.

Example:

```

java
Copy code
int[] numbers = {1, 2, 3, 4, 5}; // One-dimensional array
System.out.println(numbers[0]); // Access first element

```

19. What is the use of Packages in Java?

A **package** in Java is used to group related classes and interfaces together. It helps in organizing the code, avoiding name conflicts, and providing access protection and easier maintenance.

Example:

```

java
Copy code
package utilities; // Package declaration

public class Calculator {
    public int add(int a, int b) {
        return a + b;
    }
}

```


20. Define Swing.

Swing is a GUI toolkit in Java that is part of the Java Foundation Classes (JFC). It provides a rich set of components like buttons, labels, tables, and text fields for building window-based applications. Swing components are lightweight and platform-independent.

Example:

java

Copy code

```
import javax.swing.*;

public class MyWindow {
    public static void main(String[] args) {
        JFrame frame = new JFrame("Swing Example");
        JButton button = new JButton("Click Me");
        frame.add(button);
        frame.setSize(300, 200);
        frame.setVisible(true);
    }
}
```

21. Define ComponentEvent Class.

The **ComponentEvent** class in Java is part of the AWT (Abstract Window Toolkit) event handling mechanism. It is used for handling events related to component changes, such as resizing or moving a window.

Example:

java

Copy code

```
import java.awt.*;
import java.awt.event.*;

public class ComponentExample {
    public static void main(String[] args) {
        Frame f = new Frame("ComponentEvent Example");
```

```

        f.addComponentListener(new ComponentAdapter() {
            public void componentResized(ComponentEvent e) {
                System.out.println("Window resized");
            }
        });
        f.setSize(300, 200);
        f.setVisible(true);
    }
}

```

22. Differentiate between Swing and JPanel.

Swing is a part of Java's GUI toolkit, while **JPanel** is a container used to group components together. JPanel is often used as a child container inside a frame to organize the layout of components.

Example:

```

java
Copy code
import javax.swing.*;

public class MyPanel {
    public static void main(String[] args) {
        JFrame frame = new JFrame("Swing Example");
        JPanel panel = new JPanel();
        JButton button = new JButton("Button on Panel");
        panel.add(button);
        frame.add(panel);
        frame.setSize(300, 200);
        frame.setVisible(true);
    }
}

```

23. Discuss various steps involved in loading and running a remote applet.

The steps involved in loading and running a remote applet include:

1. The applet is stored on a remote server.
2. The applet is requested by the user through a browser.
3. The applet's .class file is downloaded to the local machine from the server.
4. The applet is loaded into the browser and executed within the applet viewer.

24. Differentiate between drawRect() and fillRect() methods.

`drawRect()` is used to draw a rectangle's outline, while `fillRect()` fills the entire rectangle with a specified color. `drawRect()` only draws the border, and `fillRect()` fills the interior.

Example:

```
java
Copy code
Graphics g = getGraphics();
g.setColor(Color.RED);
g.drawRect(50, 50, 100, 50); // Draws rectangle outline
g.fillRect(200, 50, 100, 50); // Fills rectangle with color
```

25. Write a short note on command line argument.

Command line arguments are passed to the Java program at the time of running it. They are stored in an array of `String` objects and can be used to pass data to the program from the command prompt.

Example:

```
java
Copy code
public class CommandLineExample {
    public static void main(String[] args) {
        for (String arg : args) {
            System.out.println(arg); // Prints each command line
argument
        }
    }
}
```

26. List out the different branching statements used in Java.

The main branching statements in Java are:

1. **if** - Used for conditional execution.
2. **if-else** - Executes one block if the condition is true, and another block if false.
3. **switch** - Used for selecting one of many code blocks to execute based on a variable.

27. Define a class.

A **class** is a blueprint or template in Java that defines the properties (fields) and behaviors (methods) of objects. A class is used to create instances (objects) and is fundamental to object-oriented programming.

Example:

```
java
Copy code
class Car {
    String model;

    void displayModel() {
        System.out.println(model);
    }
}
```

28. How will you access class members using objects?

Class members (fields and methods) can be accessed using an object by referencing the class instance. The dot operator (.) is used to access them.

Example:

```
java
Copy code
Car car = new Car(); // Creating an object
car.model = "Toyota"; // Accessing the class member 'model'
car.displayModel(); // Calling the method using the object
```

29. What is the use of protected keyword in Java?

The protected keyword in Java is used to specify that a member (variable or method) can be accessed within the same package or by subclasses (even if they are in different packages).

Example:

```
java
Copy code
class Animal {
    protected void sound() {
        System.out.println("Animal sound");
    }
}

class Dog extends Animal {
    void sound() {
        System.out.println("Bark");
    }
}
```

30. What is Array?

An **array** in Java is a data structure that stores multiple values of the same type in a single variable. Arrays are indexed and the elements can be accessed using their index.

Example:

```
java
Copy code
int[] numbers = {1, 2, 3, 4, 5}; // Declare and initialize an array
System.out.println(numbers[2]); // Accessing the third element (3)
```

31. Explain newborn state of a thread?

In Java, the **newborn state** (also called the "new state") is the state when a thread is created but has not yet started execution. The thread is in this state after invoking the Thread class constructor but before calling `start()`.

32. Why exception handling is done?

Exception handling is done to handle runtime errors effectively. It allows a program to continue running even if an error occurs, instead of crashing. It improves code reliability and user experience.

33. What are differences between Swing and AWT?

- **AWT (Abstract Window Toolkit)** uses native OS components, while **Swing** is lightweight and does not rely on the OS's native components.
- Swing provides more sophisticated components than AWT and is part of Java's standard GUI toolkit.

34. Discuss various sections of a web page.

A typical web page consists of the following sections:

1. **Header** - Contains the title, navigation links, and metadata.
2. **Body** - Contains the main content, such as text, images, and videos.
3. **Footer** - Contains additional information, such as copyright or contact details.

35. What is JDBC ResultSet?

The **ResultSet** in JDBC represents the result set of a database query. It provides methods to iterate through the data, retrieve values, and update rows in the result set.

Example:

```
java  
Copy code
```

```
ResultSet rs = stmt.executeQuery("SELECT * FROM users");
while (rs.next()) {
    System.out.println(rs.getString("name"));
}
```

36. Differentiate between object-oriented and procedure-oriented programming.

Object-Oriented Programming (OOP) focuses on organizing code into classes and objects. It emphasizes the use of encapsulation, inheritance, polymorphism, and abstraction. **Procedure-Oriented Programming (POP)**, on the other hand, is based on the concept of functions or procedures. Code is organized into functions, and the main focus is on the sequence of tasks or procedures to execute.

Example: In OOP, classes like Car and Person might interact with each other, whereas in POP, the program might contain functions like moveCar() or driveCar().

37. Explain the use of continue statement.

The **continue** statement in Java is used to skip the current iteration of a loop and move to the next iteration. It is useful when you want to skip specific conditions during a loop without exiting the loop.

Example:

```
java
Copy code
for (int i = 1; i <= 5; i++) {
    if (i == 3) {
        continue; // Skip printing 3
    }
    System.out.println(i);
}
```

38. Define classes and objects.

A **class** in Java is a template or blueprint for creating objects. It defines the properties (variables) and behaviors (methods) that the objects created from it will have. An **object** is an instance of a class, created using the new keyword.

Example:

java

Copy code

```
class Animal {
    String name;
    void speak() {
        System.out.println(name + " says Hello!");
    }
}

public class Main {
    public static void main(String[] args) {
        Animal a = new Animal(); // Object creation
        a.name = "Lion";
        a.speak();
    }
}
```

39. What are constructors?

A **constructor** in Java is a special method used to initialize objects. It has the same name as the class and does not have a return type. It is called automatically when an object is created using the new keyword.

Example:

java

Copy code

```
class Car {
    String brand;

    // Constructor
    Car(String b) {
        brand = b;
    }
}
```



```

    }

    void displayBrand() {
        System.out.println("Brand: " + brand);
    }
}

public class Main {
    public static void main(String[] args) {
        Car car1 = new Car("Toyota");
        car1.displayBrand();
    }
}

```

40. What is the use of protected keyword in Java?

The protected keyword in Java is used to specify that a member (variable or method) of a class can be accessed within the same package or by subclasses, even if they are in different packages. It offers more access than private but less than public.

Example:

```

java
Copy code
class Animal {
    protected void sound() {
        System.out.println("Some sound");
    }
}

class Dog extends Animal {
    void sound() {
        System.out.println("Bark");
    }
}

```

41. What is Array?

An **array** in Java is a collection of elements of the same data type stored in a contiguous memory location. It allows you to store multiple values in a single variable and access them using indices.

Example:

```
java
Copy code
int[] numbers = {10, 20, 30, 40}; // Array initialization
System.out.println(numbers[1]); // Accessing the second element
(20)
```

42. Explain newborn state of a thread?

In Java, the **newborn state** refers to the state of a thread when it is created but has not yet started running. The thread is in this state immediately after calling the Thread class constructor, but before invoking the `start()` method.

43. Why exception handling is done?

Exception handling in Java is used to manage runtime errors in a program in a structured way. It prevents the program from crashing and allows it to continue functioning, providing users with a more reliable experience. It also enables error logging and debugging.

Example:

```
java
Copy code
try {
    int division = 10 / 0; // Throws ArithmeticException
} catch (ArithmeticException e) {
    System.out.println("Error: Cannot divide by zero");
}
```

44. What are differences between Swing and AWT?

- **AWT (Abstract Window Toolkit)** is Java's original GUI toolkit, using native components, which makes it platform-dependent.
- **Swing** is a more modern GUI toolkit that provides a richer set of components and is platform-independent, as it uses lightweight components (not tied to native OS components).

45. Discuss various sections of a web page.

A typical **web page** consists of the following sections:

1. **Header:** Contains meta-information like the title, logo, and navigation links.
2. **Body:** Holds the main content of the page such as text, images, forms, and tables.
3. **Footer:** Includes additional information such as contact details, copyright, or social media links.

46. What is JDBC ResultSet?

A **ResultSet** in JDBC represents the result set of a database query. It provides methods for accessing data returned by a query, allowing you to iterate over rows and extract column values.

Example:

```
java
Copy code
ResultSet rs = stmt.executeQuery("SELECT * FROM users");
while (rs.next()) {
    System.out.println(rs.getString("username"));
}
```

47. Differentiate between numeric literal and non-numeric literal.

A **numeric literal** represents a constant value in a program, such as integers or floating-point numbers. For example, 10 or 3.14. A **non-numeric literal** represents constant values that are not numeric, such as strings, characters, and boolean values.

Example:

```
java
Copy code
int num = 10; // Numeric literal
String name = "John"; // Non-numeric literal
```

48. Differentiate between the operators = and ==.

- = is the **assignment operator**, used to assign a value to a variable.
- == is the **equality operator**, used to compare if two values are equal.

Example:

```
java
Copy code
int a = 5; // Assignment using '='
if (a == 5) { // Comparison using '=='
    System.out.println("a is equal to 5");
}
```

49. What is the use of this keyword?

The **this** keyword in Java refers to the current instance of the class. It is used to refer to the current object, especially when distinguishing between class fields and parameters with the same name.

Example:

```
java
Copy code
class Car {
    String model;

    Car(String model) {
        this.model = model; // 'this' refers to the current
object's 'model'
    }
}
```

```
}
```

50. What is constructor overloading?

Constructor overloading in Java refers to the ability to define multiple constructors with different parameter lists within the same class. This allows objects of the class to be created with different initial values.

Example:

java

Copy code

```
class Car {
    String model;

    Car() {
        model = "Unknown";
    }

    Car(String model) {
        this.model = model;
    }
}

public class Main {
    public static void main(String[] args) {
        Car car1 = new Car(); // Calls the default constructor
        Car car2 = new Car("Toyota"); // Calls the parameterized
constructor
    }
}
```

51. What is a derived class?

A **derived class** is a class that inherits properties and behaviors from a base class (also called a superclass). The derived class can add or modify methods and fields from the base class.

Example:

```
java
Copy code
class Animal {
    void eat() {
        System.out.println("Animal is eating");
    }
}

class Dog extends Animal { // Derived class
    void bark() {
        System.out.println("Dog barks");
    }
}
```

52. Define a Package.

A **package** in Java is a namespace that organizes a set of related classes and interfaces. It helps in avoiding name conflicts and makes the program easier to maintain. Java has both built-in packages and user-defined packages.

Example:

```
java
Copy code
package mypackage; // User-defined package

public class MyClass {
    public void display() {
        System.out.println("Hello from MyClass");
    }
}
```

53. What is exception handling?

Exception handling in Java is a mechanism to handle runtime errors, allowing the program to continue executing or to report the error in a controlled manner. Java provides try, catch, finally, and throw for managing exceptions.

Example:

```
java
Copy code
try {
    int result = 10 / 0; // Dividing by zero will throw an
exception
} catch (ArithmeticException e) {
    System.out.println("Error: Division by zero");
} finally {
    System.out.println("Finally block executed");
}
```

54. What is the Delegation Event Model?

The **Delegation Event Model** in Java is a way of handling events by delegating the event handling task to a specific object (usually a listener). In this model, components (like buttons) do not handle events directly but instead delegate the task to an event listener class, which processes the event.

Example:

```
java
Copy code
class ButtonClickListener implements ActionListener {
    public void actionPerformed(ActionEvent e) {
        System.out.println("Button Clicked!");
    }
}
```

Here, the `ButtonClickListener` class handles the `actionPerformed` event when a button is clicked.

55. Write a short note on JCheckBox.

A **JCheckBox** in Java Swing is a component that allows the user to select one or more options from a set of choices. It can have two states: selected and unselected. When a `JCheckBox` is selected, it indicates a positive choice, and when unselected, it indicates a negative choice.

Example:

java

Copy code

```
JCheckBox checkBox = new JCheckBox("Accept Terms and Conditions");  
checkBox.setSelected(true); // Sets the checkbox as selected
```

56. Define a Layout Manager. List different Layout Managers.

A **Layout Manager** in Java is an interface that defines how the components of a container (like a JFrame) are arranged. It is responsible for determining the size and position of components inside a container. Some of the most commonly used layout managers are:

1. **FlowLayout**: Components are arranged in a line, and once the line is filled, a new line is created.
2. **BorderLayout**: Divides the container into five sections: North, South, East, West, and Center.
3. **GridLayout**: Components are arranged in a grid with equal-sized cells.
4. **CardLayout**: Manages a set of components that can be flipped between.
5. **GridBagLayout**: A flexible layout that arranges components in a grid with varying row and column spans.

57. Distinguish between init() and start() methods of an applet.

- **init()**: It is called only once when the applet is loaded for the first time. It is used to initialize the applet and set up its environment.
- **start()**: It is called after the init() method and whenever the applet becomes visible. It is typically used for tasks that need to run when the applet is active, like animation or processing.

58. What are the JDBC API components?

The **JDBC (Java Database Connectivity)** API components allow Java programs to interact with relational databases. The key components include:

1. **DriverManager**: Manages database drivers.
2. **Connection**: Establishes a connection to the database.
3. **Statement**: Executes SQL queries.
4. **ResultSet**: Represents the result set from a query.

5. **SQLException**: Handles database-related errors.

59. Differentiate between the operators & and &&.

- **& (bitwise AND operator)**: It is a bitwise operator that operates on individual bits of two operands.
- **&& (logical AND operator)**: It is a logical operator used in conditional statements. It returns true only if both conditions are true, and it short-circuits, meaning it stops evaluating once the first condition is false.

Example:

```
java
Copy code
int a = 5, b = 10;
if (a > 0 && b > 0) { // Logical AND (&&)
    System.out.println("Both are positive");
}

int result = a & b; // Bitwise AND (&)
```

60. Write the syntax of switch case structure.

The **switch case** structure allows you to execute one of many blocks of code based on the value of a variable. It is typically used as an alternative to multiple if-else statements.

Syntax:

```
java
Copy code
switch (variable) {
    case value1:
        // Code to be executed if variable == value1
        break;
    case value2:
        // Code to be executed if variable == value2
        break;
    default:
        // Code to be executed if no case matches
```

```
}
```

61. Define classes and objects.

A **class** in Java is a blueprint for creating objects. It defines properties (fields) and behaviors (methods) that the objects created from it will have. An **object** is an instance of a class, and it holds actual values for the properties defined by the class.

Example:

```
java
```

```
Copy code
```

```
class Car {
    String color;
    void start() {
        System.out.println("Car started!");
    }
}

public class Main {
    public static void main(String[] args) {
        Car myCar = new Car(); // Object of the class Car
        myCar.color = "Red";
        myCar.start();
    }
}
```

62. Why multiple inheritance is not supported in Java?

Java does not support **multiple inheritance** (a class inheriting from more than one class) because it can lead to ambiguity. For example, if two parent classes have the same method, the child class might not know which one to inherit. Java uses **interfaces** to achieve multiple inheritance without ambiguity.

63. What is an abstract method?

An **abstract method** is a method that is declared without an implementation. It must be overridden in a subclass. Abstract methods are used in **abstract classes** to enforce that subclasses provide their own implementation.

Example:

```
java
Copy code
abstract class Animal {
    abstract void sound(); // Abstract method
}

class Dog extends Animal {
    void sound() {
        System.out.println("Bark");
    }
}
```

64. Explain one-dimensional array with example?

A **one-dimensional array** in Java is a collection of variables of the same type, stored in contiguous memory locations. Each element of the array is accessed using an index.

Example:

```
java
Copy code
int[] numbers = {1, 2, 3, 4, 5}; // One-dimensional array
System.out.println(numbers[2]); // Output: 3
```

65. What is the use of Packages in Java?

A **package** in Java is used to group related classes and interfaces together, which helps avoid class name conflicts. It also provides access control and organizes files in a project efficiently. Java has built-in packages (e.g., `java.util`, `java.io`) and user-defined packages.

66. Define Swing.

Swing is a GUI toolkit for Java that provides a richer set of components than AWT (Abstract Window Toolkit). Swing components are **lightweight**, meaning they are not dependent on the platform's native windowing system, making them platform-independent.

Example:

```
java
Copy code
JButton button = new JButton("Click Me");
```

67. Define ComponentEvent Class.

The **ComponentEvent** class in Java is part of the **java.awt.event** package. It represents events that occur when a component's size, position, or visibility changes. It is often used to track changes in components such as resizing or hiding.

68. Differentiate between Swing and JPanel.

- **Swing** is a GUI toolkit in Java that provides lightweight components and is built on top of AWT.
- **JPanel** is a specific container in Swing that is used to group multiple components together inside a window.

Example:

```
java
Copy code
JPanel panel = new JPanel(); // JPanel to group components
JButton button = new JButton("Button");
panel.add(button);
```

69. Discuss various steps involved in loading and running a remote applet.

The steps involved in loading and running a **remote applet** are:

1. **Downloading:** The applet is downloaded from a web server to the client machine.
2. **Loading:** The applet class files are loaded into the applet viewer or a browser.
3. **Initialization:** The `init()` method is called to set up the applet.
4. **Execution:** The `start()` method is invoked, and the applet starts running.
5. **Destruction:** When the applet is no longer needed, the `destroy()` method is called.

70. Differentiate between `drawRect()` and `fillRect()` methods.

- **`drawRect(x, y, width, height)`:** Draws an empty rectangle with the specified dimensions, using the current drawing color.
- **`fillRect(x, y, width, height)`:** Fills the rectangle with the current fill color.

Example:

```
java
Copy code
Graphics g = getGraphics();
g.drawRect(50, 50, 100, 60); // Draw an empty rectangle
g.fillRect(200, 50, 100, 60); // Fill a rectangle with color
```

71. Write a short note on command line argument.

Command line arguments in Java are values passed to a program when it is executed. They are accessible within the `main` method as an array of `String` values. These arguments allow users to provide input directly from the command line.

Example:

```
java
Copy code
public class CommandLine {
    public static void main(String[] args) {
        for (String arg : args) {
```

```

        System.out.println(arg);
    }
}

```

72. What are the different states of a thread in Java?

A thread in Java can exist in the following states:

1. **New:** The thread is created but not yet started.
2. **Runnable:** The thread is ready to run and waiting for the CPU to be assigned.
3. **Blocked:** The thread is waiting for a resource to become available.
4. **Waiting:** The thread is waiting indefinitely for another thread to perform a specific action.
5. **Terminated:** The thread has finished executing.

73. What is meant by 'this' reference?

The **this** reference in Java refers to the current instance of the class. It is commonly used to differentiate between instance variables and method parameters when they have the same name.

Example:

```

java
Copy code
class Car {
    String model;

    Car(String model) {
        this.model = model; // 'this' refers to the instance
variable
    }
}

```

74. What is the use of the continue statement?

The **continue** statement in Java is used inside loops to skip the current iteration and proceed with the next iteration of the loop. It is typically used when you want to skip certain conditions and continue processing the loop.

Example:

java

Copy code

```
for (int i = 1; i <= 5; i++) {  
    if (i == 3) {  
        continue; // Skip the iteration when i is 3  
    }  
    System.out.println(i);  
}
```

Output: 1, 2, 4, 5 (3 is skipped)

75. State the uses of the super keyword in Java.

The **super** keyword in Java refers to the immediate parent class of a subclass. It can be used to access:

1. Parent class methods.
2. Parent class constructors.
3. Parent class variables that are hidden by the subclass.

Example:

java

Copy code

```
class Animal {  
    void sound() {  
        System.out.println("Animal makes a sound");  
    }  
}  
  
class Dog extends Animal {  
    void sound() {  
        super.sound(); // Calls the parent class method  
        System.out.println("Dog barks");  
    }  
}
```

76. What are static variables?

A **static variable** in Java is a class-level variable shared by all instances (objects) of the class. It is initialized only once, at the start of the program, and its value is shared among all instances of the class.

Example:

```
java
Copy code
class Counter {
    static int count = 0;  // Static variable

    Counter() {
        count++;
    }
}

public class Main {
    public static void main(String[] args) {
        new Counter();
        new Counter();
        System.out.println(Counter.count);  // Output: 2
    }
}
```

77. What are interfaces?

An **interface** in Java is a reference type, similar to a class, that can contain only constants, method signatures, default methods, static methods, and nested types. Interfaces cannot have instance fields and can only provide method signatures. Classes implement interfaces to define the behavior promised by the interface.

Example:

```
java
Copy code
interface Animal {
    void sound();  // Abstract method
}
```



```

class Dog implements Animal {
    public void sound() {
        System.out.println("Bark");
    }
}

```

78. List down the Java API packages.

Java provides many **API packages** for building applications. Some of the key Java API packages are:

1. **java.lang** – Contains fundamental classes like String, Math, Object, etc.
2. **java.util** – Provides utility classes such as List, Set, Map, Date, etc.
3. **java.io** – Includes classes for input and output operations (e.g., File, BufferedReader).
4. **java.net** – Deals with networking classes like URL, Socket, etc.
5. **javax.swing** – Contains Swing classes for building GUIs.

79. Write any four methods of the Thread class.

Some key methods of the **Thread** class in Java include:

1. **start()**: Starts the thread's execution.
2. **run()**: Contains the code to be executed when the thread runs.
3. **sleep(long millis)**: Pauses the thread for a specified time.
4. **join()**: Ensures that the calling thread waits for the thread to finish execution.

Example:

java

Copy code

```

class MyThread extends Thread {
    public void run() {
        System.out.println("Thread is running");
    }
}

```

```

public class Main {
    public static void main(String[] args) {
        MyThread t = new MyThread();
    }
}

```

```
        t.start(); // Starts the thread
    }
}
```

80. What are the differences between Swing and AWT?

- **Swing** is part of the Java Foundation Classes (JFC) and provides a richer set of GUI components, including features like pluggable look-and-feel and better performance. Swing components are **lightweight** and do not rely on the platform's native GUI.
- **AWT (Abstract Window Toolkit)** is Java's original GUI library. It provides basic GUI components, but AWT components are **heavyweight**, meaning they rely on the underlying platform's windowing system.

Example:

```
java
Copy code
// Swing Example
JButton swingButton = new JButton("Swing Button");

// AWT Example
Button awtButton = new Button("AWT Button");
```

81. List the components of a Swing.

Swing provides a wide range of GUI components. Some key Swing components include:

1. **JButton**: A button that can trigger events.
2. **JLabel**: Displays a short string or an image.
3. **TextField**: Allows the user to enter a single line of text.
4. **TextArea**: Allows the user to enter multiple lines of text.
5. **ComboBox**: A drop-down list of items.
6. **CheckBox**: A checkbox that can be either selected or unselected.
7. **RadioButton**: A button in a group where only one button can be selected at a time.

Example:

java

Copy code

```
JButton button = new JButton("Click Me");
JLabel label = new JLabel("Hello, World!");
```

82. Define the term polymorphism.

Polymorphism in Java allows objects of different classes to be treated as objects of a common superclass. It is one of the core concepts of Object-Oriented Programming (OOP), enabling a single interface to represent different underlying forms (methods or objects). Polymorphism can be achieved through method overriding and method overloading.

Example:

java

Copy code

```
class Animal {
    void sound() {
        System.out.println("Animal makes a sound");
    }
}

class Dog extends Animal {
    @Override
    void sound() {
        System.out.println("Dog barks");
    }
}

public class Main {
    public static void main(String[] args) {
        Animal animal = new Dog();
        animal.sound(); // Polymorphism, calls Dog's sound()
    }
}
```

83. Explain the use of the continue statement.

The **continue** statement in Java is used to skip the current iteration of a loop and proceed to the next iteration. It is useful when a specific condition within the loop is met, and you want to skip the remaining code in that iteration.

Example:

java

Copy code

```
for (int i = 1; i <= 5; i++) {  
    if (i == 3) {  
        continue; // Skips the current iteration when i is 3  
    }  
    System.out.println(i);  
}
```

Output: 1, 2, 4, 5 (3 is skipped)

84. State the uses of the super keyword in Java.

The **super** keyword in Java refers to the immediate parent class. It can be used to:

1. Call a parent class's method.
2. Access a parent class's constructor.
3. Refer to a parent class's field that is hidden by a subclass.

Example:

java

Copy code

```
class Animal {  
    void sound() {  
        System.out.println("Animal makes a sound");  
    }  
}  
  
class Dog extends Animal {  
    void sound() {
```

```

        super.sound(); // Calling the superclass method
        System.out.println("Dog barks");
    }
}

public class Main {
    public static void main(String[] args) {
        Dog dog = new Dog();
        dog.sound();
    }
}

```

85. What are static variables?

A **static variable** in Java is shared by all instances of a class. It is a class-level variable, meaning that there is only one copy of the variable, no matter how many instances of the class are created. Static variables are initialized only once, when the class is loaded.

Example:

```

java
Copy code
class Counter {
    static int count = 0; // Static variable

    Counter() {
        count++;
    }
}

public class Main {
    public static void main(String[] args) {
        new Counter();
        new Counter();
        System.out.println(Counter.count); // Output: 2
    }
}

```

86. What are interfaces?

An **interface** in Java defines a contract for what a class can do, but not how it does it. It contains abstract methods that the implementing class must define. Interfaces can also include default methods with implementation and static methods. They allow multiple inheritance of method signatures.

Example:

```
java
Copy code
interface Animal {
    void sound(); // Abstract method
}

class Dog implements Animal {
    public void sound() {
        System.out.println("Bark");
    }
}

public class Main {
    public static void main(String[] args) {
        Dog dog = new Dog();
        dog.sound();
    }
}
```

87. List down the Java API packages.

Java provides many **API packages** that provide pre-built classes and interfaces for various tasks. Some of the key Java API packages are:

1. **java.lang** - Fundamental classes like String, Object, etc.
2. **java.util** - Utilities like collections (List, Set), date/time (Date, Calendar), etc.
3. **java.io** - Input and output classes like File, BufferedReader, etc.
4. **java.net** - Networking classes like Socket, URL, etc.

5. **javax.swing** - Swing components for building graphical user interfaces.

88. Write any four methods of the Thread class.

Some key methods of the **Thread** class are:

1. **start()**: Starts the thread by invoking its **run()** method.
2. **run()**: Contains the code that defines the thread's task.
3. **sleep(long millis)**: Pauses the thread for the specified time in milliseconds.
4. **join()**: Waits for the thread to complete before proceeding.

Example:

java

Copy code

```
class MyThread extends Thread {
    public void run() {
        System.out.println("Thread is running");
    }
}

public class Main {
    public static void main(String[] args) {
        MyThread t = new MyThread();
        t.start(); // Starts the thread
    }
}
```

89. What are the differences between Swing and AWT?

- **AWT (Abstract Window Toolkit)**: AWT is Java's original GUI toolkit and is based on the native operating system's windowing system. It provides basic GUI components, but these are **heavyweight**, meaning they rely on the operating system's resources.
- **Swing**: Swing is a more modern GUI toolkit and is **lightweight**, meaning it doesn't rely on the operating system's windowing system. It provides a richer set of UI components, such as

pluggable look-and-feel and better control over component behavior.

Example:

```
java
Copy code
// AWT Button
Button awtButton = new Button("AWT Button");

// Swing Button
JButton swingButton = new JButton("Swing Button");
```

90. List the components of a Swing.

Swing provides a comprehensive set of GUI components. Some common components include:

1. **JButton** - A button that can trigger events.
2. **JLabel** - Displays text or images.
3. **TextField** - A single-line text box for input.
4. **TextArea** - A multi-line text area for input.
5. **ComboBox** - A dropdown list of options.
6. **CheckBox** - A checkbox with two states: selected or unselected.
7. **RadioButton** - A button in a group where only one option can be selected at a time.

91. What is meant by 'this' reference?

The **this** keyword in Java refers to the current instance of the class. It is often used to differentiate between instance variables and parameters when they share the same name. It can also be used to call the current class's methods and constructors.

Example:

```
java
Copy code
class Car {
    String model;
```



```
    Car(String model) {  
        this.model = model; // 'this' refers to the current  
instance variable  
    }  
}
```

92. What are the different states of a thread in Java?

A thread in Java can exist in several states:

1. **New:** The thread is created but not started yet.
2. **Runnable:** The thread is ready to run and waiting for CPU time.
3. **Blocked:** The thread is blocked and waiting for a resource.
4. **Waiting:** The thread is waiting indefinitely for some other thread to perform a specific action.
5. **Terminated:** The thread has finished executing.